YINSON PRODUCTION - TREASURY REPORTING

FX Parities Reporting Engine - Technical Documentation

Architecture, processing logic, function inventory, data requirements, outputs, and maintenance notes

Source script	FX_PARITIES_OVERVIEW_REPORTING_ENGINE-FINAL.py
Prepared for	Operational users, maintainers, and treasury reporting stakeholders
Document purpose	Provide a professional runbook and technical understanding of the FX Parities Reporting Engine

Prepared from the current engine implementation. This document describes the behaviour that is actually present in the supplied script file.

Contents

1. Document Purpose and Scope
2. Engine Overview
3. Problems Solved and Business Value
4. Code Architecture Chapters
5. End-to-End Processing Flow
6. Operating Modes
7. Input Requirements
8. Data Model and Transformation Logic
9. Output Deliverables
10. Function Inventory
11. Non-Technical Intuition Summary
12. Configuration and Maintenance Points
13. Error Handling, Controls, and Known Limitations
14. Appendix - Key Fields and Formula Definitions

Document Purpose and Scope

This document explains what the FX Parities Reporting Engine is designed to do, how it is structured, how it processes data end to end, and how each major function contributes to the final outputs.

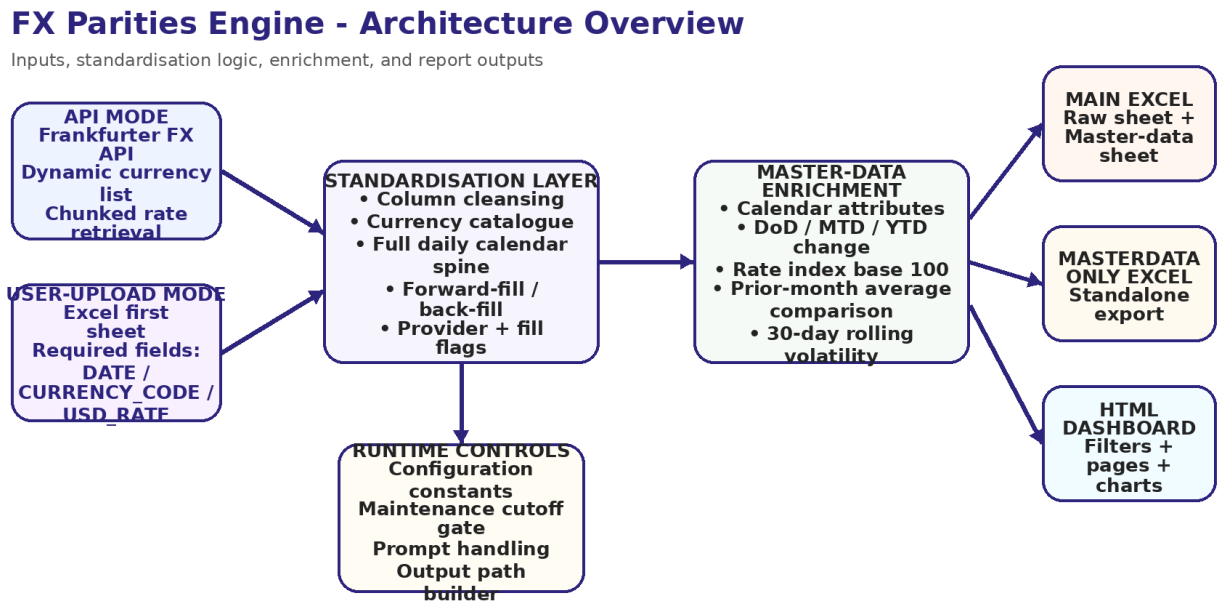
The description is based on the supplied Python implementation and is intended for both technical maintainers and non-developer business users who need a reliable understanding of the engine.

Engine Overview

The engine is a reporting pipeline that produces three deliverables in one run: a main Excel workbook, a standalone master-data workbook, and an interactive HTML dashboard.

It supports two data acquisition modes. In API mode, it fetches daily USD-based FX rates automatically from Frankfurter. In user-upload mode, it accepts an existing Excel file and reshapes it into the same standard structure used by the rest of the pipeline.

The engine therefore acts as both a data collector and a standardiser. Regardless of the source, the objective is to end with a clean, calendar-complete, analytics-ready FX master dataset.



Problems Solved and Business Value

The engine removes the manual effort of collecting daily FX rates, checking currency coverage, filling missing non-trading days, and preparing downstream files for reporting consumers.

It also solves a consistency problem. Whether the source is the API or a user-prepared workbook, the engine converts the data into a controlled raw schema and then enriches it into a standard master-data schema.

From a treasury reporting perspective, this means rate history, rate changes, calendar attributes, volatility indicators, and dashboard visuals are generated in a repeatable way instead of being rebuilt manually each reporting cycle.

Code Architecture Chapters

The script is intentionally separated into functional layers so that acquisition, shaping, export, and orchestration responsibilities remain easy to maintain.

- Configuration and runtime controls
- Terminal prompts and user interaction helpers
- API acquisition layer
- User-upload validation layer
- Calendar standardisation and master-data enrichment
- Excel export layer
- Dashboard payload and HTML generation
- File auto-open helpers
- Main orchestration and execution entry point

End-to-End Processing Flow

The engine starts by determining the local date and timestamp, applying a maintenance validity gate, and displaying a startup banner.

It then asks the user to choose the data source. The chosen branch prepares the observed dataset, after which both branches converge into the same calendar-building and master-data enrichment flow.

Finally, the script writes the Excel and HTML outputs, opens them automatically in the default applications, and waits for a final Enter key to close the process.

- Initialise local date and load timestamp.
- Apply the silent maintenance gate.
- Show startup banner and wait for user confirmation.
- Choose API mode or upload mode.
- Collect or validate the observed FX dataset.
- Build a full daily calendar by currency and apply fill logic.
- Create enriched master-data metrics.
- Write the main Excel workbook.
- Write the standalone master-data workbook.
- Build and write the HTML dashboard.
- Auto-open all generated files and show completion banner.

Operating Modes

API mode is intended for automated daily rate acquisition. The script first fetches the active currency catalogue, then retrieves rates in chunks, inserts a synthetic USD = 1.0 series, and consolidates all observed rows before standardising them.

User-upload mode is intended for cases where a team already has an FX workbook and wants to run the same downstream reporting logic. The script reads the first worksheet only, validates the required columns, normalises field types, and then pushes the data through the same standardisation logic used by API mode.

Mode / Topic	Summary
API mode	Pulls FX data from the Frankfurter API, using configuration-driven start date and chunked requests.
User-upload mode	Loads a user Excel file, validates the first sheet, and reshapes it into the engine standard.
Shared downstream path	Both modes use the same full-calendar build, enrichment logic, exports, and auto-open behaviour.

Input Requirements

API mode requires internet access to Frankfurter and uses the configured START_DATE as the earliest extraction point.

User-upload mode requires an Excel workbook whose first sheet contains at least DATE, CURRENCY_CODE, and USD_RATE. Optional fields are supported and defaulted when missing.

Column	Requirement	Notes
DATE	Required	Report date per row; converted to a date value.
CURRENCY_CODE	Required	ISO currency code; uppercased and trimmed.
USD_RATE	Required	Numeric USD-based FX rate.
PROVIDER	Optional	Defaults to USER_UPLOAD when missing.
IS_FILLED	Optional	Converted to boolean; unrecognised values are treated as False.
LOAD_TIMESTAMP	Optional	Defaults to the current execution timestamp when missing.

Important implementation note: in upload mode the script adds USD to the currency catalogue if it is not present, but it does not synthesise a USD = 1.0 series the way API mode does. If uploaded data does not contain USD observations, that currency will still be skipped during the full-calendar build.

Data Model and Transformation Logic

The engine first builds a raw dataset with one row per date per currency, including provider, fill flag, and load timestamp. It then enriches that raw dataset into an analytics-ready master-data table.

The master-data layer adds calendar attributes and calculation fields that support dashboarding and period-over-period analysis.

Master-data field	Definition
RATE_CHANGE_DOD_PCT	Current USD_RATE / prior day USD_RATE - 1
RATE_CHANGE_MTD_PCT	Current USD_RATE / first USD_RATE of the same month - 1
RATE_CHANGE_YTD_PCT	Current USD_RATE / first available USD_RATE for that currency - 1
RATE_INDEX_100	Current USD_RATE / first available USD_RATE * 100
PRIOR_MONTH_AVG_RATE	Average USD_RATE for the previous month for the same currency
CHANGE_VS_PRIOR_MONTH_AVG_PCT	Current USD_RATE / PRIOR_MONTH_AVG_RATE - 1
ROLLING_30D_VOLATILITY_PCT	30-day rolling standard deviation of daily rate_change_dod_pct, minimum 5 observations

Fill logic matters: missing calendar days are filled forward and, if necessary, backward. This ensures every tracked currency has a complete daily time series between the selected start and end dates.

Output Deliverables

The engine creates three user-facing deliverables per run and saves them to the FX_PARITIES_OUTPUT directory.

Deliverable	File naming pattern	Purpose
Main Excel workbook	FX_PARITIES_RATES_YYYYMMDD.xlsx	Contains raw daily FX data plus the enriched master-data sheet.
Standalone master-data workbook	FX_PARITIES_MASTERDATA_YYYYMMDD.xlsx	Contains only the master-data export sheet for downstream use.
HTML dashboard	FX_PARITIES_DASHBOARD_YYYYMMDD.html	Interactive multi-page FX dashboard with filters, charts, rankings, and heatmaps.

The main workbook contains two sheets: FX_PARITIES_DATA and FX_PARITIES_MASTERDATA. Both are formatted as branded Excel tables with fixed schemas and styling constants defined in the script.

The HTML dashboard provides date filters, view aggregation options, page navigation, KPI cards, rate cards, line charts, ranking bars, and a monthly heatmap.

Function Inventory

The following inventory lists every function present in the supplied engine and summarises its responsibility. The functions are grouped by the structural sections defined inside the script.

Maintenance Requirement Date

Controls compliance validity and prevents outdated builds from running silently beyond the cutoff window.

Function	Responsibility
_check_maintenance_gate()	Silent compliance gate tied to the maintenance cutoff date.

Terminal Helpers

Handles status logging, prompt flows, dates, timestamps, and output file path construction.

Function	Responsibility
info()	Print an informational status message to stdout.
warn()	Print a warning message to stdout.
done()	Print a completion message to stdout.
fail()	Print an error message to stdout.
local_today()	Return the current local calendar date (timezone-aware).
local_timestamp()	Return the current local datetime as a formatted string (YYYY-MM-DD HH:MM:SS).
build_output_paths()	Construct the three output file paths for a given report date.
print_banner()	Print the startup banner to the terminal and wait for the user to press Enter.
finish_banner()	Print the completion banner listing all generated output files.
ask_yes_no()	Prompt the user with a yes/no question and return a

	boolean result.
ask_existing_file_path()	Prompt the user for a file path and validate that the file exists.

Api Helpers

Handles HTTP session creation, retry logic, currency discovery, chunking, and rate acquisition from the external API.

Function	Responsibility
session_with_retries()	Create and return a requests.Session with appropriate request headers.
get_json()	Perform a GET request and return the parsed JSON response.
normalize_currencies_payload()	Normalise the currencies endpoint response into a two-column DataFrame.
fetch_active_currencies()	Fetch the list of currently available currencies from the Frankfurter API.
chunked()	Split a list into successive sub-lists of at most `size` elements.
fetch_rate_chunk()	Fetch USD-based FX rates for a single chunk of currencies over a date range.
fetch_observed_rates_api()	Fetch all observed (non-filled) FX rates from the API for the given date range.

User-Upload Helpers

Validates, cleans, and standardises user-supplied Excel input.

Function	Responsibility
normalize_columns()	Standardise column names to uppercase with underscores replacing spaces.
normalize_bool_series()	Convert a mixed-type Series into a boolean Series.
load_user_excel_raw()	Load and validate the user-provided Excel file.
build_currency_catalog_from_upload()	Build a minimal currency catalogue from the codes present in a user upload.

Shaping Helpers

Transforms source data into a complete calendar dataset and an enriched master-data model.

Function	Responsibility
build_full_calendar_dataset()	Expand observed trading-day rates to a full daily calendar with forward-fill.
build_masterdata()	Enrich the raw FX dataset into the dashboard-ready master-data table.

Excel Helpers

Writes branded, structured Excel workbooks and tables.

Function	Responsibility
to_excel_value()	Convert a pandas or Python value to a type safe for writing into openpyxl.
auto_width()	Set each column's width to fit its widest cell value, clamped between 12 and 42.
style_sheet()	Apply the standard Yinson treasury styling to a

	worksheet.
add_dataframe_as_table()	Write a DataFrame into a worksheet starting at row 4 and register it as
prepare_output_dataframe()	Prepare a DataFrame for Excel export by uppercasing column names and
write_workbook()	Create and save the main two-sheet Excel workbook.
write_master_only_workbook()	Create and save a standalone single-sheet workbook containing only the

Dashboard Payload / Html

Converts master-data into a JSON payload and then into an interactive HTML dashboard.

Function	Responsibility
build_dashboard_payload()	Serialise the master-data DataFrame into a JSON-compatible payload dict
dashboard_html()	Render the complete self-contained HTML dashboard by injecting the data
write_dashboard()	Render and write the HTML dashboard to disk.

Open Helpers

Launches generated files in the operating system default applications.

Function	Responsibility
open_file()	Open a file in the system's default application.

Main Orchestration

Runs the end-to-end business process from mode selection to output delivery.

Function	Responsibility
prepare_data_via_api()	Fetch FX data from the Frankfurter API and prepare all output artefacts.
prepare_data_via_upload()	Load a user-provided Excel file and prepare all output artefacts.
choose_data_mode()	Ask the user whether to use API mode or user-upload mode, then delegate
main()	Top-level orchestration function.

Non-Technical Intuition Summary

Intuitively, this engine behaves like an FX data factory. It takes either a live feed or a user workbook, checks the quality of what it receives, fills the missing calendar days, calculates useful analytics, and packages the results into three different reporting formats.

A non-technical user can think of the pipeline in four stages: get the data, make the data complete, make the data meaningful, and publish the data in ready-to-use outputs.

Configuration and Maintenance Points

The script exposes most operational behaviour through a clearly visible configuration block. This is where date range, workbook branding, dashboard focus coverage, and network settings are controlled.

For maintainability, downstream documentation should always be updated when any user-facing constant, prompt, schema, or output naming pattern is changed.

Configuration item	Why it matters
START_DATE	Earliest date fetched in API mode. Currently fixed to 2026-01-01.
API_BASE	Frankfurter API base endpoint.
OUTPUT_DIR	Folder where all generated outputs are saved.
RAW_SHEET / MASTER_SHEET	Worksheet names used in the Excel exports.
TITLE_BG / HEADER_BG / BODY_BG / TAB_COLOR	Workbook styling constants for branded Excel output.
REQUEST_TIMEOUT / CHUNK_SIZE / SLEEP_BETWEEN_CALLS	Network and batching behaviour for API execution.
FOCUS_CURRENCIES	Currencies prioritised on the Executive Radar dashboard page.
OTHER_PAGE_SIZE	How many non-focus currencies are grouped on each extra dashboard page.
RAW_COLUMNS / MASTER_COLUMNS	Controlled output schema for raw and enriched exports.
STATIC_CURRENCY_NAMES	Fallback mapping from ISO code to display name.
YES_VALUES / NO_VALUES	Accepted responses for the yes/no prompt.
_MAINTENANCE_CUTOFF	Compliance cutoff after which the silent gate becomes active.

Error Handling, Controls, and Known Limitations

The engine includes several resilience features: retry logic for HTTP requests, validation for yes/no inputs and file paths, duplicate removal, missing value handling, and automatic warnings when a currency cannot be processed.

It also includes a silent maintenance cutoff check. After the configured cutoff, the process gives a short acknowledgement window and exits if no confirmation is received. This is a compliance-oriented control rather than a reporting feature.

There are also practical implementation limitations that a maintainer should know.

- The dashboard is described as self-contained, but the HTML template still references Google Fonts and Chart.js via external CDNs. That means true offline rendering is not fully guaranteed in the current implementation.
- API mode creates a synthetic USD = 1.0 series. Upload mode does not; it relies on whatever data is actually present in the upload file.
- The upload path reads the first worksheet only. If the user places the correct dataset on another sheet, the script will not use it.
- The start date for API mode is hard-coded in the configuration block rather than supplied interactively.
- The engine uses local system time for report date and load timestamp, so results may differ across machines if their local dates or timezones differ.

Appendix - Key Fields and Formula Definitions

The raw dataset schema contains DATE, CURRENCY_CODE, USD_RATE, PROVIDER, IS_FILLED, and LOAD_TIMESTAMP.

The master-data schema extends the raw schema with calendar attributes and analytics used by the dashboard and downstream analysis.

Output layer	Key fields / structure
Raw dataset	DATE, CURRENCY_CODE, USD_RATE, PROVIDER, IS_FILLED, LOAD_TIMESTAMP
Master-data	Raw fields plus calendar attributes, rate change metrics, rate index, prior-month context, and rolling volatility
Dashboard pages	Executive Radar page plus chunked “Other Currencies” pages based on focus list and page size configuration